



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-46433 lldpd: Heap OOB Read in VLAN Decapsulation memmove

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-15
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-46433 lldpd: Heap OOB Read in VLAN Decapsulation memmove - Vulnerability Report

Published: 2026-06-15 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

CVE-2026-46433 lldpd: Heap OOB Read in VLAN Decapsulation memmove

1. Executive Summary

CVE-2026-46433 is a medium-severity adjacent-network vulnerability in `lldpd`, an open-source implementation of IEEE 802.1AB Link Layer Discovery Protocol (LLDP). Prior to version `1.0.22`, `lldpd_decode()` in `src/daemon/lldpd.c` removes an 802.1Q VLAN tag by shifting the frame payload four bytes left with `memmove()`, but the byte count passed to `memmove()` fails to subtract those four bytes. When a VLAN-tagged LLDP frame exactly fills the interface MTU allocation, the daemon reads four bytes beyond the heap buffer.

The practical impact is availability-focused. Public scoring from GitHub/CNA and NVD rates the flaw `CVSS:3.1 6.5 / MEDIUM` with adjacent-network attack vector, no privileges, no user interaction, and high availability impact. The CVE text describes a four-byte heap over-read rather than attacker-controlled write or direct remote code execution. Defenders should nevertheless treat the issue seriously on switches, virtualization hosts, appliances, Linux servers, and management segments where LLDP is enabled and untrusted or semi-trusted hosts can transmit Layer 2 frames on the same broadcast domain.

The vulnerability is fixed upstream in `lldpd` `1.0.22` by commit `ca931be63a9cae0fcd8e9b6ae4e916d49f141cd6`, which changes a single `memmove()` length expression from `s - 2 * ETHER_ADDR_LEN` to `s - 2 * ETHER_ADDR_LEN - 4`. No reviewed public source provided evidence of CISA KEV inclusion or confirmed broad exploitation at the time of this report; CISA's CVE 5.0 ADP enrichment marks exploitation as `none`, automatable as `no`, and technical impact as `partial`. Immediate priorities are patching, inventorying LLDP-enabled assets, segmenting management networks, and adding crash/packet telemetry around VLAN-tagged LLDP frames.

2. Intelligence Requirement

This report answers the following intelligence requirement: what does CVE-2026-46433 mean for defenders operating `lldpd` on servers, hypervisors, appliances, and management networks, which exposure patterns are materially risky, and what detection, hunting, and remediation actions should be prioritized now?

Question	Assessment
What is the flaw?	A four-byte heap out-of-bounds read in <code>lldpd_decode()</code> during VLAN decapsulation of received Ethernet frames.
What is the vulnerable primitive?	<code>memmove(frame + 2 * ETHER_ADDR_LEN, frame + 2 * ETHER_ADDR_LEN + 4, s - 2 * ETHER_ADDR_LEN)</code> uses a byte count four bytes too large.
Who can reach it?	An attacker able to send crafted VLAN-tagged LLDP frames on the same Layer 2 segment as an interface where <code>lldpd</code> receives frames.
What is the likely impact?	Daemon crash, process instability, or denial of service; public sources do not describe confidentiality or integrity impact.
What versions are affected?	<code>lldpd</code> before <code>1.0.22</code> ; GitHub advisory lists vulnerable range <code><= 1.0.21</code> and patched version <code>1.0.22</code> .
What should defenders do first?	Upgrade to <code>1.0.22</code> or a vendor backport, then confirm LLDP exposure boundaries and monitor for <code>lldpd</code> crashes or suspicious VLAN-tagged LLDP traffic.
Is exploitation confirmed in the wild?	No confirmed broad exploitation was identified in reviewed public sources; CISA ADP currently marks exploitation as <code>none</code> .

3. Source Review and Confidence

Source	Contribution	Confidence
GitHub security advisory `GHSA-2g8p-2h3j-63m3`	Primary public advisory with vulnerable code snippet, data flow, affected range, patched version, CVSS, and impact summary	High
CVE JSON 5.2 CNA record	Canonical CVE metadata, affected versions, CWE-125, CVSS vector, and CISA ADP enrichment	High
NVD JSON 2.0 entry	Confirms analyzed status, CPE range, public description, CVSS v3.1 score, weakness, and references	High
Upstream patch commit `ca931be63a9cae0fcd8e9b6ae4e916d49f141cd6`	Direct engineering evidence that the remediation is a one-line correction to the `memmove()` length	High
Upstream PR `lldpd/lldpd#787`	Confirms the fix was merged on 2026-05-12 under the title “Fix heap OOB read in VLAN decapsulation memmove”	High
`lldpd` `1.0.22` release notes / `NEWS`	Confirms release inclusion: “Fix out-of-bound read access when removing VLAN tag (CVE-2026-46433, #787)”	High
CISA Known Exploited Vulnerabilities catalog	Checked for KEV inclusion; no `CVE-2026-46433` match in the 2026-06-12 catalog snapshot	Medium-High
Downstream issue and package activity, including XCP-ng PR `xcp-ng-rpms/lldpd#2`	Shows downstream packaging awareness and planned security update activity	Medium

Confidence is high on flaw mechanics, affected version boundary, and remediation because the reviewed sources include the CNA/GitHub advisory, NVD enrichment, release notes, merged PR, and exact patch commit. Confidence is moderate on exploitation prevalence: the reviewed public record does not show KEV inclusion, named campaign use, or exploit-kit integration, but absence of public exploitation reporting should not be treated as proof that local Layer 2 testing or opportunistic misuse is impossible.

Two analytic caveats are important. First, the intake item entered the workflow through the MSRC update guide, but the underlying product is upstream `lldpd`, not a Microsoft-authored component. Microsoft and other vendors may track the issue because they package `lldpd` in downstream products or distributions. Second, the attack vector is adjacent network; this reduces internet-scale exposure but increases relevance inside flat campus networks, lab environments, virtualization fabrics, appliance management networks, and shared hosting segments where untrusted hosts can emit LLDP Ethernet frames.

4.1 Vulnerability metadata

Field	Value
CVE	`CVE-2026-46433`
Product	`lldpd`
Component	`lldpd_decode()` in `src/daemon/lldpd.c`

Vulnerability class	`CWE-125: Out-of-bounds Read`
Public severity	`CVSS v3.1 6.5 / MEDIUM`
CVSS vector	`CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
Attack vector	Adjacent network / Layer 2
Required privileges	None
User interaction	None
Affected versions	`lldpd` `< 1.0.22` / advisory range `<= 1.0.21`
Fixed version	`lldpd` `1.0.22`
Patch commit	`ca931be63a9cae0fcd8e9b6ae4e916d49f141cd6`
Upstream release date	`1.0.22` published 2026-05-14
CVE publication date	CNA publication 2026-06-09; NVD publication 2026-06-09
Exploitation status	No reviewed evidence of KEV inclusion or broad in-the-wild exploitation

4.2 Flaw mechanics

The vulnerable code path strips a VLAN tag from a received Ethernet frame before normal LLDP decoding. The destination pointer starts at offset `2 \* ETHER\_ADDR\_LEN`, the source pointer starts four bytes later to skip the VLAN tag, and the frame length `s` is reduced by four bytes after the move. The bug is that the `memmove()` byte count uses the full remaining length after the two Ethernet addresses instead of the remaining length minus the four bytes being skipped.

```
struct ether_header eheader;
memcpy(&eheader, frame, sizeof(struct ether_header));
if (eheader.ether_type == htons(ETHERTYPE_VLAN)) {
    memmove(frame + 2 * ETHER_ADDR_LEN, frame + 2 * ETHER_ADDR_LEN + 4,
            s - 2 * ETHER_ADDR_LEN);    // vulnerable: should subtract 4
    s -= 4;
}
```

The public advisory describes the triggering data flow as follows: `lldpd\_recv()` allocates a heap buffer sized to the hardware MTU, receives a frame that may be as large as that MTU, then calls `lldpd\_decode(cfg, buffer, n, hardware)`. If the attacker-supplied frame is VLAN-tagged and the received size equals the MTU, the source range used by `memmove()` extends four bytes past the heap allocation. The upstream patch changes the third argument to `s - 2 \* ETHER\_ADDR\_LEN - 4`, aligning the copy length with the four-byte left shift.

4.3 Exposure profile

Exposure pattern	Why it matters	Priority
LLDP enabled on server, hypervisor, appliance, or network-management interfaces attached to a shared Layer 2 segment	Adjacent hosts may be able to transmit crafted LLDP/VLAN frames to the vulnerable daemon	Immediate
Virtualization hosts or cloud/private-cloud fabrics using LLDP for topology discovery	Tenant, lab, or compromised guest/network-adjacent systems may share the relevant Layer 2 domain depending on architecture	High
Campus or OT/IoT segments where	Physical or rogue-device adjacency can	High

unmanaged devices can join switch ports	become a practical trigger path	
Downstream distributions or appliances shipping `lldpd` `<= 1.0.21` without a backported fix	Package version strings may not obviously map to upstream `1.0.22`; vendor advisory state must be checked	High
Environments that do not run `lldpd`, block inbound LLDP frames, or strictly isolate discovery traffic to trusted switch infrastructure	Practical exposure is materially reduced	Reduced

4.4 Defensive significance

CVE-2026-46433 is not a remote internet-facing vulnerability in the usual web-service sense. The vulnerable traffic is Layer 2 LLDP, which normally stays on the local link and is often used for topology discovery rather than application delivery. That constraint limits mass scanning but does not make the issue irrelevant. Internal attackers, rogue devices, compromised hosts, malicious lab systems, or misconfigured virtual networks may all be able to transmit adjacent-network frames where `lldpd` listens.

The most plausible operational outcome is a denial-of-service condition against the `lldpd` process or associated service health. A crash may degrade network-discovery visibility, appliance management, inventory collection, topology mapping, or automation that depends on LLDP neighbor state. In some environments, repeated process instability can also create alert fatigue or obscure more serious lateral-movement activity occurring on the same segment.

4.5 Indicator assessment

Indicator category	Assessment
Stable malicious IP/domain indicators	None identified; the vulnerable protocol is adjacent-network LLDP rather than routable internet infrastructure.
File hash indicators	None identified; this is a vulnerable-code condition, not a malware sample.
Best network indicators	VLAN-tagged LLDP frames near MTU size, repeated LLDP frames from unexpected MAC addresses, or abnormal LLDP traffic on segments where only switches should speak LLDP.
Best host indicators	`lldpd` crash loops, segmentation faults, service restarts, core dumps, or package inventory showing `lldpd` below the fixed boundary.
Best exposure indicators	LLDP enabled on untrusted access ports, server NICs, hypervisor uplinks, or management VLANs without source filtering or network admission controls.

5. Threat Context

LLDP is a discovery protocol designed for local-link topology exchange. Many organizations enable `lldpd` on Linux servers, appliances, hypervisors, network monitoring nodes, or specialized infrastructure so they can learn switch-port, VLAN, chassis, and neighbor metadata. That same operational convenience creates a local attack surface when discovery frames from untrusted or semi-trusted systems are accepted by a daemon written in memory-unsafe C.

The bug's adjacent-network scope makes it most relevant after an adversary has gained local network presence. Examples include a compromised workstation on an enterprise access VLAN, a malicious device attached to an under-controlled switch port, a guest or lab system on a virtual network where LLDP reaches the host or peer systems, or an attacker who has pivoted

into an infrastructure management segment. For red-team and incident-response planning, CVE-2026-46433 is best framed as a local disruption or visibility-degradation primitive rather than an initial access vulnerability.

Reviewed public reporting did not identify named threat actors or public exploit chains using this CVE. CISA ADP enrichment in the CVE record lists `Exploitation: none`, `Automatable: no`, and `Technical Impact: partial`, and the KEV catalog check did not return a match. Defenders should still avoid relying solely on exploitation status because the vulnerable primitive is simple, the patch is public, and Layer 2 abuse often leaves less centralized telemetry than web exploitation attempts.

Scenario	Description	Defensive implication
Rogue device on an access switch	A device sends crafted VLAN-tagged LLDP frames at MTU size toward hosts running vulnerable `lldpd`	Enforce port security, disable unneeded LLDP receive, and alert on LLDP from non-infrastructure OUIs or MACs
Compromised server on a flat management VLAN	An attacker with foothold emits LLDP traffic to disrupt discovery daemons on adjacent systems	Segment management networks and monitor service restarts across the same L2 domain
Virtual lab or private-cloud adjacency	A guest, test VM, or tenant-adjacent system can reach discovery traffic paths unexpectedly	Validate LLDP forwarding behavior in virtual switches and hypervisor uplink designs
Downstream package lag	A product or distro ships a vulnerable `lldpd` package after upstream fixed `1.0.22`	Track vendor backports and verify the actual source patch, not just the upstream version number

6. Detection and Hunting

Detection should combine exposure management, network packet visibility, and host/service health telemetry. There is no stable IOC list to block. The strongest signals are vulnerable package inventory, LLDP traffic patterns that should not exist in the environment, and `lldpd` crashes correlated with VLAN-tagged LLDP frames or unexpected MAC addresses.

6.1 Detection coverage summary

Signal	Telemetry source	Analytic goal	Caveat
`lldpd` package below `1.0.22` or vendor package without the backport	EDR inventory, vulnerability scanner, package manager, SBOM	Identify assets requiring patch validation	Vendor backports may fix the bug without changing to upstream `1.0.22`
`lldpd` crash, core dump, or repeated restart	systemd journal, syslog, EDR process telemetry, crash collectors	Detect possible successful denial-of-service attempts	Benign malformed frames or unrelated bugs can produce similar crashes
VLAN-tagged LLDP frames near interface MTU	Packet capture, Zeek custom analyzer, Suricata/pcap, switch SPAN	Surface potential trigger traffic	Many networks do not centrally inspect Layer 2 discovery traffic
LLDP frames from non-switch endpoints or unexpected MAC OUIs	Switch telemetry, NAC, packet capture, EDR/NDR	Identify rogue or compromised adjacent devices	Requires an environment-specific allowlist of legitimate LLDP speakers
Burst of LLDP frames followed by service instability	Correlated network + host telemetry	Connect suspicious local-link traffic to impact	Clock skew and lack of packet capture can reduce confidence

6.2 Sigma - lldpd crash or restart activity on Linux

title: lldpd Crash or Repeated Restart After CVE-2026-46433 Disclosure

```

id: 2d522c83-3765-4f7a-9d09-b4d2a2dcfbf1
status: experimental
description: Detects lldpd crash, core dump, segmentation fault, or repeated restart telemetry that may
indicate malformed LLDP/VLAN frame handling, including possible CVE-2026-46433 attempts.
author: Lost Boys Cyber
date: 2026-06-15
references:
  - https://github.com/lldpd/lldpd/security/advisories/GHSA-2g8p-2h3j-63m3
  - https://github.com/lldpd/lldpd/commit/ca931be63a9cae0fcd8e9b6ae4e916d49f141cd6
logsource:
  product: linux
  service: syslog
detection:
  selection_lldpd:
    - Image|endswith: '/lldpd'
    - ProcessName: 'lldpd'
    - Message|contains: 'lldpd'
  selection_failure:
    Message|contains:
      - 'segfault'
      - 'core dumped'
      - 'Main process exited'
      - 'Failed with result'
      - 'restart counter'
      - 'SIGSEGV'
  condition: selection_lldpd and selection_failure
falsepositives:
  - Non-security lldpd defects or administrative service restarts
  - Test environments intentionally fuzzing LLDP parsers
level: medium
tags:
  - attack.t1499

```

6.3 QQL - Defender for Endpoint service crash/restart hunt

```

let lookback = 30d;
DeviceProcessEvents
| where Timestamp > ago(lookback)
| where FileName in~ ('lldpd', 'systemd', 'systemctl', 'coredumpctl')
| where ProcessCommandLine has_any ('lldpd', 'core dumped', 'restart', 'SIGSEGV', 'segfault')
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName,
InitiatingProcessCommandLine, ReportId
| order by Timestamp desc

```

6.4 QQL - vulnerable lldpd package inventory review

```

let lookback = 14d;
DeviceTvmSoftwareInventory
| where TimeGenerated > ago(lookback)
| where SoftwareName has_any ('lldpd', 'lldpd daemon', 'lldpd')
| project TimeGenerated, DeviceName, SoftwareName, SoftwareVersion, SoftwareVendor, OSPlatform
| extend NeedsReview = iff(SoftwareVersion matches regex @"^(0\.|1\.\0\.(0|1|2|3|4|5|6|7|8|9|1[0-9]|20|
21)([^\0-9].*)?$)", 'review-version-or-backport', 'verify-vendor-state')
| order by NeedsReview desc, DeviceName asc

```

6.5 SPL - syslog and systemd review for lldpd instability

```

(index=os OR index=linux OR index=syslog)
("lldpd" AND ("segfault" OR "core dumped" OR "Main process exited" OR "restart counter" OR "SIGSEGV" OR
"Failed with result"))
| stats earliest(_time) as first_seen latest(_time) as last_seen count by host source sourcetype
process message
| convert ctime(first_seen) ctime(last_seen)
| sort - count

```


6.6 Suricata rule - VLAN-tagged LLDP frame size review

The following rule is an analyst starting point for packet-capture environments that can inspect local-link Ethernet traffic. Tune the `dsize` threshold to local MTU and reduce noise with interface, VLAN, or MAC allowlists.

```
alert ether any any -> any any (msg:"LBC Potential CVE-2026-46433 VLAN-tagged LLDP oversized frame review"; ether_proto:0x8100; content:"|88 cc|"; offset:16; depth:2; dsize:>1450; classtype:attempted-dos; sid:4643301; rev:1; metadata: cve CVE-2026-46433, service lldp;)
```

6.7 Hunt questions

Hypothesis	What to prove or disprove
Vulnerable `lldpd` exists on assets attached to untrusted Layer 2 segments	Join package inventory with network-zone and switch-port metadata; prioritize assets on access, lab, shared, or management VLANs.
Non-infrastructure endpoints are sending LLDP or VLAN-tagged LLDP frames	Review SPAN/packet-capture, switch telemetry, or NAC data for LLDP sources outside approved switch/hypervisor OUIs.
Recent `lldpd` instability correlates with suspicious local-link traffic	Correlate systemd/coredump events with LLDP frame bursts, MAC-address anomalies, or port-security events within a short time window.
Vendor appliances ship a vulnerable downstream package	Review appliance SBOMs, support bundles, and vendor advisories for `lldpd` version or backport evidence.

7. Recommendations

Priority	Owner	Action	Target timeline
Critical	Infrastructure / Linux platform owners	Upgrade `lldpd` to `1.0.22` or a vendor-supported package containing commit `ca931be63a9cae0fcd8e9b6ae4e916d49f141cd6`.	7 days for exposed management/server segments; next standard cycle for low-risk isolated assets
High	Vulnerability management	Inventory `lldpd` installations, including appliances and hypervisors, and map each host to network zone, LLDP receive status, and package source.	7-14 days
High	Network engineering	Restrict LLDP reception to trusted infrastructure paths where possible; review access-port policies, NAC controls, and rogue-device detection.	14 days
Medium	SOC / detection engineering	Deploy the included service-instability detections and add packet-capture or switch-telemetry hunts for unexpected LLDP speakers.	14-30 days
Medium	Incident response	Add `lldpd` crashes and suspicious LLDP bursts to triage playbooks for management-network disruption or rogue-device	30 days

		investigations.	
Medium	Asset owners	If patching is delayed, disable `lldpd` where not operationally required or configure it to limit receive behavior on untrusted interfaces.	Until patched

Defenders should validate vendor backports carefully. Some distributions may ship a version string below upstream `1.0.22` while carrying the one-line fix, while others may retain a vulnerable package until their security update is published. Where version ambiguity exists, confirm the presence of the corrected `memmove()` length expression in source packages or vendor advisory notes.

Network controls are especially valuable because the issue requires Layer 2 adjacency. Limit where LLDP is accepted, prevent untrusted endpoints from transmitting discovery traffic into management segments, and alert when workstations, guests, or non-network devices speak LLDP. These controls reduce exposure not only to CVE-2026-46433 but also to future parser flaws in discovery daemons.

8. References

- [R1] GitHub Security Advisory, `GHSA-2g8p-2h3j-63m3`, “Heap OOB Read in VLAN Decapsulation memmove” — <https://github.com/lldpd/lldpd/security/advisories/GHSA-2g8p-2h3j-63m3>
- [R2] CVE Program CVE JSON 5.2 record for `CVE-2026-46433` — <https://cveawg.mitre.org/api/cve/CVE-2026-46433>
- [R3] NVD, `CVE-2026-46433` — <https://nvd.nist.gov/vuln/detail/CVE-2026-46433>
- [R4] Upstream patch commit `ca931be63a9cae0fcd8e9b6ae4e916d49f141cd6` — <https://github.com/lldpd/lldpd/commit/ca931be63a9cae0fcd8e9b6ae4e916d49f141cd6>
- [R5] Upstream pull request `lldpd/lldpd#787`, “Fix heap OOB read in VLAN decapsulation memmove” — <https://github.com/lldpd/lldpd/pull/787>
- [R6] `lldpd` release `1.0.22` — <https://github.com/lldpd/lldpd/releases/tag/1.0.22>
- [R7] `lldpd` `NEWS` file for `1.0.22` — <https://raw.githubusercontent.com/lldpd/lldpd/1.0.22/NEWS>
- [R8] CISA Known Exploited Vulnerabilities catalog, checked 2026-06-15; no `CVE-2026-46433` match in catalog version dated 2026-06-12 — <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- [R9] XCP-ng downstream package PR `xcp-ng-rpms/lldpd#2`, “Enable testsuite & Fix CVE-2026-46433” — <https://github.com/xcp-ng-rpms/lldpd/pull/2>
- [R10] MSRC update guide entry for `CVE-2026-46433` — <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-46433>